Mid-Semester Project Report

Centralized Campus Food Ordering System

Lab and Group: P2 Group-18

GitHub: https://github.com/daemon1769/campus-food-ordering-system/

1. Project Objective and Problem Statement:

- **The Problem:** Campus food courts suffer from long queues, fragmented cash payments, and no centralized way for students to track their spending or daily nutrition.
- **The Solution:** We are building a full-stack, cashless database application that manages food ordering, financial ledgers, and dietary tracking using a centralized database system.
- **Core System Users:**
    - **Students (Consumers):** Place cashless orders using a pre-paid semester ledger. They can query the system to track their daily and weekly calorie, protein, and financial data.
    - **Outlets (Vendors):** Receive real-time orders, manage their inventory, and use aggregation queries to view their income analytics.
    - **Food Safety Union (Admins):** Insert new outlets into the system, control the master menu catalogue, and monitor system-wide data.

2. Application Design and Architecture:

- **Proposed Technology Stack:**
    - **Frontend:** React + Vite (creates a responsive, component-based user interface for the three distinct portals).
    - **Backend:** Node.js with Express.js (handles business logic, session states, and REST API requests).
    - **Database:** PostgreSQL (chosen to enforce strict relational integrity constraints, foreign keys, and ACID transactions).
- **High-Level Architecture:**
    - The system follows a standard 3-tier decoupled architecture.
    - The React frontend sends user actions to the backend server.
    - The backend validates business rules and executes SQL operations on the PostgreSQL relational database, ensuring secure data retrieval and modification.

3. Core Application Workflows:

- **Workflow 1: Union Setup and Auditing**
    - The Union creates new food outlets in the system and standardizes their specific menus.
    - They define every item's attribute, including base price, category (e.g., rice, tiffin, juice), dietary type (veg/non-veg), parcel surcharges, and exact nutritional values (calories, protein).

    - The Union can run administrative queries to generate Excel-style breakdowns of any student's total semester expenditure or any outlet's total revenue.

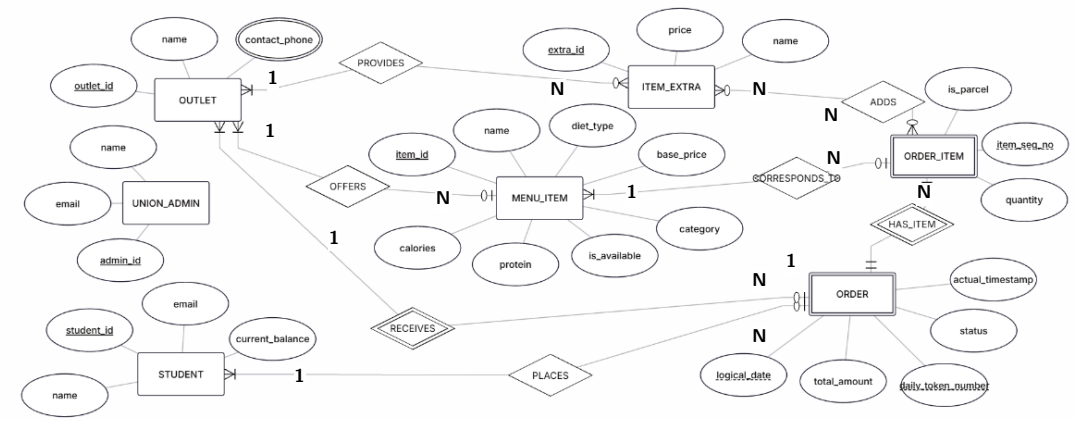
- **Workflow 2: Student Ordering and Tracking**
    - Students log in to view their current semester ledger balance alongside their cumulative spending and nutritional intake, which can be filtered by day, week, or month.
    - They browse an outlet's menu, viewing only items currently marked "available." They can filter items by category or dietary preference (with neutral add-ons like extra cheese appearing in both veg and non-veg filters).
    - The cart dynamically calculates total cost (including parcel fees) and total nutrition.
    - **Order Logic & Constraints:** A student can place simultaneous orders at different outlets, but the database restricts them from placing a new order at the *same* outlet until their previous order is marked "Ready." Students can cancel an order as long as it is in the "Pending" state.
    - Upon ordering, the student receives a unique **Daily Token Number** (e.g., #101). This sequence is unique per outlet and resets daily based on the logical business day (6 AM to 4 AM).
- **Workflow 3: Outlet Fulfilment and Analytics**
    - Vendors can instantly toggle the live availability of their menu items.
    - Outlets receive incoming "Pending" orders and choose to Accept or Reject them.
    - **Delayed Financial Transaction:** The database executes an ACID transaction to deduct the funds from the student's central balance *only* at the exact moment the outlet clicks "Accept."
    - Once prepared, the outlet updates the order status to "Ready."
    - Outlets access advanced reporting dashboards powered by grouping and aggregation queries, allowing them to view daily, weekly, and monthly income, complete with category-wise and item-wise revenue breakdowns.

4. Business Rules and constraints:

- **Semester Financial Cycle (System Setup):** At the start of each semester, each student's ledger is credited with an initial deposit (e.g., ₹12,000). The current_balance attribute is permitted to decrease into negative values to accurately represent accumulated debt. The database strictly models the active semester, with end-of-semester debts exported externally for fee collection before the ledger resets.
- **The "Logical Day" Boundary (Time Setup):** A standard operational business day is defined as running from 6:00 AM to 3:59 AM the following calendar day. All daily analytics, income groupings, and order token numbering reset based on this logical boundary rather than midnight.
- **Menu Item Visibility (Browsing Phase):** Students can only view and select menu items that the outlet has currently toggled as "available." Items toggled as "unavailable" by the vendor are entirely hidden from the student ordering interface.
- **Dietary Classification of "Extras" (Filtering Phase):** Base menu items are strictly classified as either vegetarian or non-vegetarian. However, specific "Item Extras" (e.g., cheese, butter, onions) are classified as dietary-neutral. These will visibly populate as add-on options regardless of whether the student is filtering the menu for vegetarian or non-vegetarian items.

- **Concurrent Ordering Restriction (Ordering Phase):** A student is prohibited from placing a new order at a specific outlet if they currently possess an active order (status "Pending" or "Accepted") at that exact same outlet. They must wait until the existing order reaches "Ready," "Rejected," or "Cancelled." They are, however, permitted to place simultaneous orders across different outlets.
- **Unique Daily Token Generation (Post-Order Phase):** Order numbers (e.g., #101) are not globally unique database IDs. They are sequentially generated per outlet, per logical day. This means a student may receive token #101 at Outlet A and #101 at Outlet B on the exact same day.
- **Order Cancellation Policy (Pending Phase):** A student may cancel an order directly from their dashboard, immediately voiding it, provided the outlet has not yet moved the status out of the "Pending" state.
- **Delayed Financial Deduction (Fulfilment Phase):** When a student places an order, the total order amount is **not** immediately deducted from their semester ledger. The financial deduction strictly occurs via a secure database transaction at the exact moment the outlet changes the order status to "Accepted."

5. ER Diagram:



6. Relational Schema:

Relation	Its Attributes
STUDENT	**student_id**, name, email, current_balance
OUTLET	**outlet_id**, name
OUTLET_PHONE	**outlet_id**, contact_phone
MENU_ITEM	**item_id**, outlet_id, name, base_price, category, diet_type, is_available, is_parcelable, parcel_charge, calories, protein

Relation	Its Attributes
ITEM_EXTRA	**extra_id**, outlet_id, name, price
UNION_ADMIN	**admin_id**, name, email
ORDER	**outlet_id**, **logical_date**, **daily_token_number**, student_id, actual_timestamp, status, total_amount
ORDER_ITEM	**outlet_id**, **logical_date**, **daily_token_number**, **item_seq_no**, item_id, quantity, is_parcel
ORDER_ITEM_EXTRA	**outlet_id**, **logical_date**, **daily_token_number**, **item_seq_no**, **extra_id**

Foreign keys:

OUTLET_PHONE.outlet_id -> OUTLET.outlet_id

MENU_ITEM.outlet_id -> OUTLET.outlet_id

ITEM_EXTRA.outlet_id -> OUTLET.outlet_id

ORDER.outlet_id -> OUTLET.outlet_id

ORDER.student_id -> STUDENT.student_id

ORDER_ITEM.(outlet_id, logical_date, daily_token_number)

-> ORDER.(outlet_id, logical_date, daily_token_number)

ORDER_ITEM.item_id -> MENU_ITEM.item_id

ORDER_ITEM_EXTRA.extra_id -> ITEM_EXTRA.extra_id

ORDER_ITEM_EXTRA.(outlet_id, logical_date, daily_token_number, item_seq_no)

-> ORDER_ITEM.(outlet_id, logical_date, daily_token_number, item_seq_no)

7. Normalization and Design Justification:

The schema is normalized to Third Normal Form (3NF) to ensure data integrity and eliminate redundancy or anomalies.

- **1NF Compliance:** Multi-valued attributes were removed by extracting contact_phone into a separate OUTLET_PHONE relation, ensuring all attributes remain atomic.
- **2NF Compliance:** Partial dependencies were eliminated. Non-prime attributes (e.g., quantity, is_parcel) depend entirely on their full composite primary keys, not just fragments.
- **3NF Compliance:** Transitive dependencies were removed. Every non-prime attribute depends exclusively on its primary key, ensuring updates occur in only one location.

- **M:N Resolution:** The many-to-many relationship was resolved by creating the associative entity ORDER_ITEM_EXTRA, cleanly bridging ORDER_ITEM and ITEM_EXTRA.
- **Weak Entity Logic:** ORDER and ORDER_ITEM inherit outlet_id. Combined with logical_date and daily_token_number, this ensures system-wide uniqueness while allowing order tokens to sequentially reset daily per outlet.

8. Current Progress:

For the mid-semester submission, our entire team collaborated extensively to establish the foundational database architecture and system requirements. Together, we defined the core problem statement, finalized the business rules, and formulated the logical day boundaries alongside the cashless ledger transaction logic. As a group, we designed the ER/EER model, drafted the initial entity definitions, and systematically mapped the relational schema to ensure strict 3NF compliance. We also collectively documented the weak entity constraints, outlined the high-level system architecture for this report, and initialized the project's GitHub repository.

9. Proposed Future Work Plan:

Team Member	Post Mid-Semester Future Work
Singireddy Hrushikesh Reddy [2024AAPS0239H]	Develop the Node.js/Express backend, configure the PostgreSQL database, and write the core SQL queries (CTEs, joins).
Kartikey Gaur [2024AAPS0225H]	Build the React + Vite frontend interfaces (Student, Outlet, and Union portals) and integrate REST APIs.
Deeptimahanti Aashrith [2024AAPS0189H]	Implement advanced database triggers (delayed ledger deductions) and format the final analytical reporting dashboards.
Aryaman Ray [2024AAPS0227H]	Lead backend API testing, manage session states, and implement error handling for concurrent order requests.
Kothapalli Manobhiram Reddy [2024AAPS0240H]	Oversee frontend-to-backend integration, refine UI/UX responsiveness, and manage final deployment testing.

10. References Used:

Class Notes: *CS F212 Database Systems - Lecture 16 Self-Study Guide* (ER/EER Diagram Design and Conversion to Relational Schema).

Modelling Tool: ERDPlus (https://erdplus.com) – Utilized for drafting and generating the Entity-Relationship (ER) diagram.

GeeksforGeeks: "Database Normalization in DBMS" (https://www.geeksforgeeks.org/database-normalization-in-dbms/) – Consulted for self-study on 1NF, 2NF, and 3NF functional dependencies.